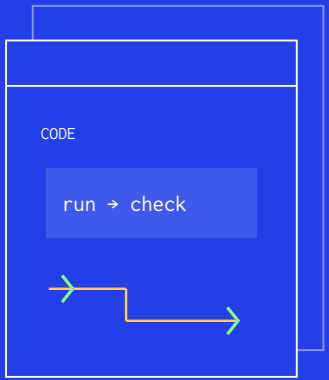
WEEK 2 / LAB 3 / TEAM WORK

MLOps: Machine Learning Operations

Same code.
Clean environment.

Lock the dependencies, package your baseline in a container you write, and rerun it away from your laptop. Needs Lab 2 merged on main (or develop).



Done means a teammate can rerun it without you

Three outcomes you can point to in the team repository.

1 **Locked**

Dependencies come from the committed uv.lock; a stale or missing lock fails visibly instead of being regenerated.

2 **Packaged**

A Dockerfile and .dockerignore you wrote build an image that runs the Lab 2 baseline and its tests.

3 **Rerun**

A fresh container reproduces the Lab 2 baseline from unchanged inputs; any difference is observed and explained.

Who does what

The split is yours; every member owns something and reviews something.

INDIVIDUAL

One PR each

One area: lock, Dockerfile, rerun or README. One branch, one PR.

TEAM

One baseline

Agree areas, merge order and the commit to package. Final rerun by a non-author.

REVIEW

Another member

Review a PR you did not write: rebuild from the README and leave a real finding.

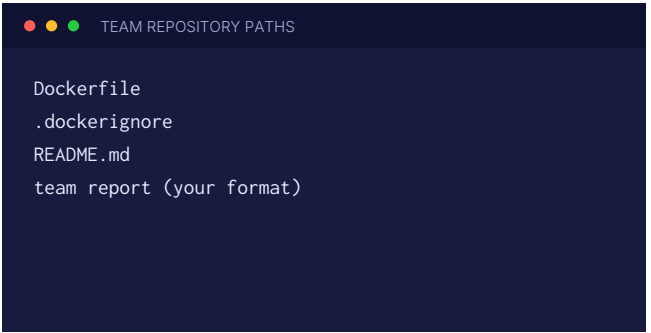
Individual report reports/week-2.md covers Labs 3–4 and lives in your personal repository.

Add these to the team repository

Paths are the contract; contents are your design.

REQUIRED

Merge them into your integration branch (main or develop) through reviewed PRs. Nothing here may need your laptop's .venv, caches or secrets.



README: how to build and rerun. Report: your team chooses its format; link raw logs and the commit.

New here: a Dockerfile for a uv project

Lecture 3 showed FROM, COPY, RUN and CMD. Each line below is a decision you make and justify.

DECISIONS, NOT ANSWERS

The lock should drive the install, and the build context should hold only what the image needs. Write it, build it, then break it on purpose.

DOCKERFILE CHECKLIST

```
FROM    pinned Python 3.12 + uv
COPY    manifest + lock first - why?
RUN     install exactly the lock
COPY    code, config, data snapshot
CMD     command that runs the baseline
.dockerignore: what never enters?
working directory: where?
```

Examples and a command cheat sheet follow; they use a different project on purpose.

Example: a Dockerfile for a small uv project

A toy app called hello; your files, paths and command will differ.

ILLUSTRATIVE EXAMPLE

Start from a base image, copy the manifest and lock, install exactly the lock, then copy code. Add a WORKDIR first so files land in /app.

Dockerfile

```
FROM python:3.12-slim
WORKDIR  /app
COPY --from=ghcr.io/astral-sh/uv:0.12 /uv /bin/
COPY pyproject.toml uv.lock ./
RUN uv sync --locked --no-install-project
COPY src ./src
RUN uv sync --locked
CMD ["uv", "run", "python", "-m", "hello"]
```

Read it, then adapt it to your project; do not paste it.

Dockerfile: three habits that matter

Condensed from the Docker build best practices and the uv Docker guide.

ORDER

Stable layers first

Base, lock and install before source, so the cache survives code edits.

COMMAND

Exec form

CMD ["prog", "arg"]. ENTRYPOINT is the main program; CMD its defaults.

CONTEXT

.dockerignore

Keep .venv, .git, caches and .env out of the build context.

Pin the base image tag; pin a digest for fully reproducible builds.

Example: what a .dockerignore keeps out

One pattern per line; paths are relative to the build context.

ILLUSTRATIVE EXAMPLE

Anything listed is not sent to the build, so it cannot leak into the image or invalidate the cache.

.dockerignore

```
.venv
.git
__pycache__
.pytest_cache
artifacts
tracking
.env
```

Decide for your project: which data, config or model files must be inside the image?

Cheat sheet: docker build and run

An image is built once; each run creates a new container from it.

BUILD AND START

Tag what you build, name what you want to inspect later, publish ports only to your own machine.

docker

```
$ docker build -t name:tag .
$ docker run --rm name:tag
$ docker run --name c1 name:tag
$ docker run -p 127.0.0.1:8000:8000 name:tag
$ docker run --network none name:tag
$ docker run -e name=value name:tag
```

--rm deletes the container on exit. -p host:container. --network none cuts the container's network at run time only.

Cheat sheet: look, copy out, clean up

Keep a named container until its outputs are safely copied out.

INSPECT AND CLEAN

Read logs first; copy results out before you remove the container; remove images you no longer need.

docker

```
$ docker ps -a
$ docker logs c1
$ docker cp c1:/app/artifacts ./out
$ docker exec -it c1 sh
$ docker image inspect name:tag
$ docker rm c1
$ docker image rm name:tag
```

exec needs a running container. image inspect shows the image ID to record as evidence.

Cheat sheet: uv lock, sync, run

The lock is the source of truth; commands differ in how strict they are.

PROJECT COMMANDS

--locked fails if the lock is stale. --frozen skips that check, so it is not a repair.

uv

```
$ uv add httpx
$ uv add --dev pytest
$ uv lock --check
$ uv sync --locked
$ uv sync --locked --no-dev
$ uv run --locked pytest -q
```

Commit pyproject.toml and uv.lock together. In an image, --no-install-project installs only dependencies.

Rules that keep the rerun honest

Fixed for every team; everything else is your choice.

1 **Clean build, no host state**

Build from a clean checkout; no bind-mounted source, .venv or earlier outputs.

2 **Unchanged inputs**

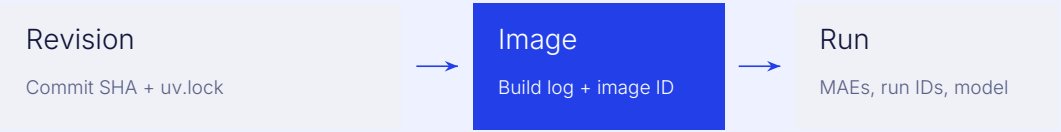
Same data snapshot, config and split as Lab 2. No tuning, API, cloud or test-set scoring.

3 **A stale lock must stop**

On a throwaway copy, show that a missing or stale lock fails. Never regenerate it to get green.

Link the result to what produced it

One chain from commit to observed metrics.



`A screenshot alone does not connect the three.`

Keep raw logs your report can link to; at most three screenshots. Compare with Lab 2 and explain differences.

Review, blockers and handover

What a reviewer checks and what an honest record contains.

REVIEW

Rebuild from README

Follow the README only; check ignore rules, locked install, evidence.

BLOCKED

Name the step

Failing step, exact message, next owner. Never hide a check.

HANDOVER

Merge, then Lab 4

Merge after review and checks. Lab 4 builds on this image.

Local runs are not remote CI; keep them as separate claims.

EXIT CHECK

Could a teammate rebuild it without asking you?

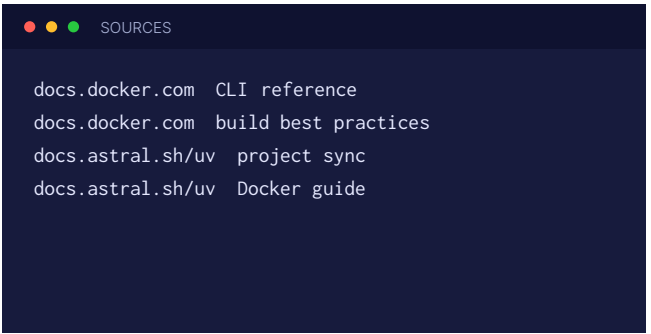
Next: Lab 4 serves the evaluated model through an API on this image. m2 is tagged after Lab 4.

Where the examples come from

Condensed from official documentation; read the originals for every option.

REFERENCES

Commands are shortened and regrouped for this lab. Check the docs when a flag is unclear.



Illustrative examples; run them in your own project before you rely on them.